Saksi — a confidential holder register for tokenized real-world assets

Live saksi-gilt.vercel.app · **Repo** github.com/PugarHuda/saksi · **Monad testnet, chain ID 10143** · Cleanverse CVI + CVA. Every figure below is printed by a command in this repo; the chain overrules any document.

Problem — measured, not asserted

Census (`ops/measure-register.mjs`, block 52181672): 562 credentialed wallets against all 104 A-Tokens on Monad, 45 of which have holders. **Median holders per asset: 3; 35 of the 45 have fewer than five; 16 have one wallet above 90% of supply.** An anonymity set of three is not anonymity, and it is structural rather than a quiet-testnet artefact: eligibility restricts the population by design, so the tighter the holder rule, the smaller the crowd its holders hide in. A privacy pool fixes that by destroying what made the asset legitimate: the issuer can no longer prove who holds what, cap concentration, or find a revoked holder.

Solution

Positions are commitments moving by JoinSplit, so amounts and owners are never published. Entry is gated twice, the exit re-checks Cleanverse on three addresses, and a regulator registers a question on-chain *before an answer exists*, then gets a proof answering exactly that: exact, threshold, range, aggregate. Private in the middle, accountable at both edges.

CVI · CVA integration points

- **Gate one is Cleanverse's contract, called inside our transaction.** `deposit()` calls `complianceVerify(pool, msg.sender)` on the CVI Compliance Validator `0xaC7e5179C2C7f03f209136886c172eb34F161792`; refusal reverts `ValidatorRefused`, and an unregistered pool reverts rather than returning `false`, so the register fails closed. `transact()` calls it three times more — sender, recipient, relayer — their model requiring a credential on every CVA recipient.
- **The asset is a real CVA.** `SAKSIAZEV`, "Saksi Series A Note", 6 decimals, issued through `atoken/launch` at `min_tier 30`, rule registered through `validator/register`. `activeRules()` forwards to `getRulesV2(pool)` and returns their live `RuleV2 (0x0000, 0x0000, 30, 0, false, 0)` = (allowedGroup, allowedSubGroup, minTier, minSubTier, isBlackList, countryBitmap) — six fields, read live from theirs. Their token-level gate is separate and also enforced: `verify_apass` answers about the A-Token itself (`atoken/rules` min_tier 30, not paused), so a wallet can pass one and fail the other.
- **Gate two's association set is derived from live CVI every build, never curated.** `ops/asp.mjs build` pages `query_apass_list`, then re-reads each known wallet through `query_apass` (`status` is null on most list rows, authoritative only per-wallet). Admits on *status null or 1, tier at least 30, unexpired*; leaf = `keccak256(wallet) mod p`, depth-10 Poseidon tree. **524 admitted of 602 queried**, root `20523674...4604905` — byte-for-byte the pool's live `aspRoot()`.
- **Revocation is a rebuild, not a blacklist.** `update_status` freezes an A-Pass, the next set is built without that leaf, and the holder can no longer prove entry; `asp.public.json` publishes the dropped leaf and the `previousRoot` it superseded.
- **Two gates, because they diverge in time rather than in trust.** A frozen credential fails gate one at once; the anchored root still admits until the issuer rebuilds. `ops/gate-gap.mjs` re-asks `validator/verify` about **all 524 members** and reports the direction of any disagreement, because a derived set must never be more permissive than its registry; a rate-limited read counts as unknown, so the figure is a floor — run it, don't quote it.
- **A third control neither gate provides.** The entry circuit's 11 public signals are `[aspRoot, denyList[8], sourceKey, commitment]`; on the live pool `denyList[0]` = `5685657930...380979` = `sourceKeyOf(0x...dEaD)`, a sanctions entry both Cleanverse gates admit and only this refuses. The proof is pinned to `msg.sender` and the exact commitment.
- **ERC-3643 shape.** `isVerified`, `canTransfer` and `canTransferWithReason` compose all three controls with an ERC-1066 status byte, so an integrator learns *which* refused. Written and tested, **not deployed** — redeploying resets the open audit requests.

Deployed — Monad testnet, chain ID 10143

SaksiPool `0xeBBA114d9870c98250239aCaFbcccc4dA09AF1CA` · **our CVA** `0xb9c53B57Cd47Bd3b55143647BeF8297d1C5f4d6B` · **Cleanverse CVI Compliance Validator** `0xaC7e5179C2C7f03f209136886c172eb34F161792` · **auditor** `0x63CB403b716111c249d4D11312c15c5744CcC4e4`, not the issuer key · six Groth16 verifiers beside the pool (addresses in `deployment.json`). Verify it yourself: `cast call <pool> "isEligible(address)(bool)" <wallet>` on `https://testnet-rpc.monad.xyz` returns Cleanverse's verdict, forwarded by the pool.

State at block 52289375: 12 commitments, 6 nullifiers spent, **6 live positions backing 2,315.000000 SAKSIAZEV**; three JoinSplits, one redeeming 50 out through a credentialed recipient and relayer; deposits from three credentialed EOAs. Eight audit requests, posted by an auditor key distinct from the issuer, **every request block earlier than its answer** (52210266 then 52210275; 52210345 then 52210355, disclosing 459.9); two are unanswerable and stay open — an aggregate over four positions summing to 1,680, asked whether they were at most 1,000. The register cannot answer falsely, only fail to answer. **173 Foundry tests pass**; `node ops/evidence.mjs` reprints it.

Scalability, priced not named (`ops/gas.mjs`, Monad's estimator on proofs this register submitted): Groth16 verification costs **940,504 gas plus 31,313 per public signal**, five points within 1,092 gas. Merkle depth is free — a private-witness dimension, so depth 20 buys a million leaves at that price — and a wider aggregate costs 62,625 gas per position.

Limits, one line: deposit amounts are public and this register's derivable from a since-fixed 37/63 split; the exit is gated operationally, not cryptographically; one aggregate answered over 3 of 4 live positions stands corrected in `audit-log.json`; the trusted setup is one operator, one machine — full list in `SUMMARY.md`.